

Learning Quantitative Finance in Rust

A Progressive Journey from Kaggle to Quant Research

Marcelo Correa

July 22, 2026

[bitcrafts/quant-lab-rust](https://github.com/bitcrafts/quant-lab-rust)

The market can stay irrational
longer than you can stay solvent.

– John Maynard Keynes

Preface

This book documents a learning journey: from data science fundamentals to quantitative finance, implemented entirely in Rust. The goal is not just to understand the concepts, but to internalize them through implementation.

Philosophy We start with beginner-friendly Kaggle projects and progressively build toward advanced quant research tools. No black-box libraries — the math is hand-rolled, because the implementation IS the learning.

Structure The book is organized in five parts:

- ▶ **Part I — Foundations:** First Kaggle projects, basic stats
- ▶ **Part II — Core Concepts:** Returns, volatility, backtesting
- ▶ **Part III — Quantitative Methods:** Moments, time series, stationarity
- ▶ **Part IV — Derivatives and Portfolio:** Options, Markowitz, factors
- ▶ **Part V — Advanced Topics:** Microstructure, AFML techniques

How to use Each chapter has:

- ▶ Learning objectives at the start
- ▶ Mathematical foundations with TikZ diagrams
- ▶ Rust implementation with code listings
- ▶ Margin notes with key formulas and insights
- ▶ Exercises and references

The companion code lives at <https://github.com/bitcrafts/quant-lab-rust>. Clone and run examples with:

```
1 git clone https://github.com/bitcrafts/quant-lab-rust.git
2 cd quant-lab-rust
3 cargo run -p qf-01-fraud --example fraud_analysis
```

Contents

List of Figures

List of Tables

FOUNDATIONS

Foundations: Mathematics, Statistics, and Programming

0

This chapter establishes the mathematical, statistical, and programming foundations needed to understand the rest of the book. If you are already comfortable with these topics, feel free to skip ahead and use this chapter as a reference.

0.1 Why These Foundations Matter

Quantitative finance sits at the intersection of three disciplines:

- **Mathematics:** The language of precision—formulas, proofs, and rigorous reasoning
- **Statistics:** The science of uncertainty—data analysis, probability, and inference
- **Programming:** The tool of implementation—turning ideas into working systems

You don't need to be an expert in all three to start. This book teaches by doing—each concept is introduced when needed and immediately applied.

The Quant Mindset

A quantitative approach means making decisions based on data and models, not intuition or emotion.

0.2 Mathematical Notation

Throughout this book, we use standard mathematical notation. Here's a quick reference:

0.2.1 Sets and Numbers

Symbol	Meaning
$\mathbb{R}$	Real numbers (e.g., 3.14, -2.5 , $\sqrt{2}$)
$\mathbb{R}^+$	Positive real numbers
$\mathbb{Z}$	Integers ($\dots, -2, -1, 0, 1, 2, \dots$)
$\mathbb{N}$	Natural numbers ($1, 2, 3, \dots$)
$\in$	"is an element of" (e.g., $x \in \mathbb{R}$ means x is real)
$\forall$	"for all"
$\exists$	"there exists"

0.2.2 Summation and Products

The summation symbol Σ means "add up":

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$$

For example, if $x = [2, 5, 3]$, then $\sum_{i=1}^3 x_i = 2 + 5 + 3 = 10$.

Summation

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$$

Product

$$\prod_{i=1}^n x_i = x_1 \times x_2 \times \dots \times x_n$$

The product symbol $\prod$ means “multiply together”:

$$\prod_{i=1}^n x_i = x_1 \times x_2 \times \cdots \times x_n$$

0.2.3 Functions and Calculus

Symbol	Meaning
$f(x)$	Function f evaluated at x
$f'(x)$ or $\frac{df}{dx}$	Derivative of f with respect to x
$\frac{\partial f}{\partial x}$	Partial derivative (when f has multiple variables)
$\ln(x)$	Natural logarithm (base $e \approx 2.718$)
$\exp(x)$ or e^x	Exponential function
$\operatorname{argmax}_x f(x)$	Value of x that maximizes f
$\operatorname{argmin}_x f(x)$	Value of x that minimizes f

Key Insight

The natural logarithm $\ln$ is everywhere in finance. Key property: $\ln(a \times b) = \ln(a) + \ln(b)$. This turns multiplication (compounding) into addition, making calculations much easier.

0.3 Statistics Essentials

Statistics helps us understand and quantify uncertainty—essential when dealing with financial markets.

0.3.1 Measures of Central Tendency

The **mean** (average) is the sum divided by the count:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The **median** is the middle value when data is sorted. It’s more robust to outliers than the mean.

0.3.2 Measures of Spread

Variance measures how spread out the data is:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Variance

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Standard deviation is the square root of variance:

$$\sigma = \sqrt{\sigma^2}$$

Standard deviation has the same units as the original data, making it easier to interpret than variance.

```

1 /// Calculate mean of a slice
2 pub fn mean(data: &[f64]) -> f64 {
3     if data.is_empty() { return 0.0; }
4     data.iter().sum::<f64>() / data.len() as f64
5 }
6
7 /// Calculate variance of a slice
8 pub fn variance(data: &[f64]) -> f64 {
9     if data.is_empty() { return 0.0; }
10    let m = mean(data);
11    data.iter().map(|x| (x - m).powi(2)).sum::<f64>() / data.len() as f64
12 }
13
14 /// Calculate standard deviation
15 pub fn std_dev(data: &[f64]) -> f64 {
16     variance(data).sqrt()
17 }

```

0.3.3 Probability Distributions

A **probability distribution** describes how likely different outcomes are.

Normal Distribution (Gaussian)

The normal distribution is the famous “bell curve”:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean and σ is the standard deviation.

We write $X \sim \mathcal{N}(\mu, \sigma^2)$ to say “ X follows a normal distribution with mean μ and variance σ^2 .”

Normal PDF

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Warning

Financial returns are NOT normally distributed! They have “fat tails”—extreme events happen much more often than the normal distribution predicts. We explore this in Chapter ??.

Uniform Distribution

Every outcome in a range is equally likely:

$$f(x) = \frac{1}{b-a} \quad \text{for } a \leq x \leq b$$

0.3.4 Expected Value and Variance

The **expected value** $\mathbb{E}[X]$ is the average outcome you’d expect over many trials:

For a discrete random variable:

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

Expected Value

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

(discrete)

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x) dx$$

(continuous)

Key properties:

- ▶ $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$ (linearity)
- ▶ $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ (additivity)
- ▶ $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

0.4 Financial Concepts

Before diving into quantitative methods, let's establish some fundamental financial concepts.

0.4.1 What is a Stock?

A **stock** (or share) represents partial ownership of a company. When you buy a stock, you own a small piece of that company and are entitled to a share of its profits.

The **stock price** is determined by supply and demand in the market. It reflects what buyers are willing to pay and sellers are willing to accept.

Stock Price

The price at which buyers and sellers agree to trade ownership.

0.4.2 OHLCV Data

Market data is typically recorded as **OHLCV**:

- ▶ **Open**: First price of the trading period
- ▶ **High**: Highest price during the period
- ▶ **Low**: Lowest price during the period
- ▶ **Close**: Last price of the period
- ▶ **Volume**: Number of shares traded

This data captures the essential price action within any time frame (day, hour, minute).

0.4.3 Returns

A **return** measures how much an investment gained or lost:

Simple return:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Log return (continuously compounded):

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Simple Return

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Log Return

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Log returns have a useful property: they add up over time.

$$r_{t_1 \rightarrow t_n} = r_{t_1} + r_{t_2} + \cdots + r_{t_n}$$

0.4.4 Risk and Volatility

Risk in finance is typically measured by **volatility**—the standard deviation of returns.

Higher volatility means more uncertainty—prices swing more wildly.

Annualized volatility scales daily volatility to a yearly figure:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$$

(252 is the typical number of trading days per year.)

Volatility

$$\sigma = \text{std}(r_t)$$

Often annualized: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$

0.4.5 The Risk-Return Tradeoff

Key Insight

Higher expected returns generally require accepting higher risk. There is no free lunch in finance—if someone promises high returns with low risk, be skeptical.

The **Sharpe ratio** measures risk-adjusted return:

$$\text{Sharpe} = \frac{\mathbb{E}[R] - R_f}{\sigma}$$

where R_f is the risk-free rate (e.g., Treasury bills).

0.5 Programming in Rust

This book uses Rust for all implementations. Here's a quick introduction for readers unfamiliar with the language.

0.5.1 Why Rust?

- **Performance:** As fast as C/C++, suitable for high-frequency trading
- **Safety:** The compiler catches bugs before runtime
- **Modern:** Great tooling, package management, and community
- **Learning:** Writing correct Rust forces you to think carefully—great for learning

0.5.2 Basic Syntax

```

1 // Variables (immutable by default)
2 let x = 5;
3 let mut y = 10; // mutable
4 y = 20;
5
6 // Functions
7 fn add(a: i32, b: i32) -> i32 {
8     a + b // no semicolon = return value
9 }
10
11 // Structs (like classes)
12 struct Stock {
13     symbol: String,
14     price: f64,
15 }
```

```

16
17 // Implementation block
18 impl Stock {
19     fn new(symbol: &str, price: f64) -> Self {
20         Stock {
21             symbol: symbol.to_string(),
22             price,
23         }
24     }
25
26     fn value(&self, shares: u32) -> f64 {
27         self.price * shares as f64
28     }
29 }
30
31 // Using the struct
32 let aapl = Stock::new("AAPL", 150.0);
33 let portfolio_value = aapl.value(100);

```

0.5.3 Traits (Interfaces)

Traits define shared behavior:

```

1 // Define a trait
2 trait Analyzer {
3     fn analyze(&self, data: &[f64]) -> f64;
4 }
5
6 // Implement for a struct
7 struct MeanAnalyzer;
8
9 impl Analyzer for MeanAnalyzer {
10     fn analyze(&self, data: &[f64]) -> f64 {
11         data.iter().sum::<f64>() / data.len() as f64
12     }
13 }
14
15 // Use generically
16 fn run_analysis<A: Analyzer>(analyzer: &A, data: &[f64]) -> f64 {
17     analyzer.analyze(data)
18 }

```

Traits

Rust's way of defining shared behavior. Similar to interfaces in Java or protocols in Swift.

This book uses traits extensively to create modular, reusable components.

0.5.4 Error Handling

```

1 // Result type for operations that can fail
2 fn divide(a: f64, b: f64) -> Result<f64, String> {
3     if b == 0.0 {
4         Err("Division by zero".to_string())
5     } else {
6         Ok(a / b)
7     }
8 }
9
10 // Using Result
11 match divide(10.0, 2.0) {
12     Ok(result) => println!("Result: {}", result),
13     Err(e) => println!("Error: {}", e),
14 }
15
16 // Or use the ? operator
17 fn calculate() -> Result<f64, String> {
18     let x = divide(10.0, 2.0)?; // propagates error
19     Ok(x * 2.0)
20 }

```

0.5.5 Iterators

Rust's iterators are powerful and efficient:

```

1 let prices = vec![100.0, 102.0, 101.0, 105.0];
2
3 // Map and collect
4 let returns: Vec<f64> = prices.windows(2)
5     .map(|w| (w[1] - w[0]) / w[0])
6     .collect();
7
8 // Filter
9 let positive: Vec<f64> = returns.iter()
10     .filter(|&r| r > 0.0)
11     .copied()
12     .collect();
13
14 // Fold (reduce)
15 let sum: f64 = prices.iter().sum();
16 let product: f64 = prices.iter().product();

```

0.6 Running the Examples

All code examples in this book are available in the companion repository.

0.6.1 Prerequisites

1. Install Rust: <https://rustup.rs/>
2. Clone the repository:

```

1 git clone https://github.com/bitscrafts/quant-lab-rust.git
2 cd quant-lab-rust

```

0.6.2 Running Examples

Each chapter has a corresponding crate with examples:

```

1 # Chapter 1: Credit Card Fraud
2 cargo run -p qf-01-fraud --example fraud_analysis
3
4 # Chapter 2: Loan Default
5 cargo run -p qf-02-loan --example loan_analysis
6
7 # Chapter 3: Stock Prices
8 cargo run -p qf-03-stocks --example stock_analysis

```

0.6.3 Running Tests

```

1 # Run all tests
2 cargo test
3
4 # Run tests for a specific crate
5 cargo test -p qf-01-fraud

```

0.7 Summary

This chapter covered:

- **Mathematical notation:** Summation, products, functions, logarithms
- **Statistics:** Mean, variance, standard deviation, distributions
- **Financial concepts:** Stocks, OHLCV, returns, volatility, Sharpe ratio

- **Rust programming:** Variables, functions, structs, traits, error handling

Next chapter: We apply these foundations to detect credit card fraud using anomaly detection.

Hello Finance: Credit Card Fraud Detection

1

Companion Code:

```
qf-01-fraud Code: crates/qf-01-fraud/  
Dependencies: qf-common  
Run: cargo run -p qf-01-fraud -example fraud_analysis
```

1.1 Learning Objectives

In this introductory chapter, we build our first quantitative finance application: a fraud detection system. Along the way, we learn:

- ▶ How to load and parse financial CSV datasets in Rust
- ▶ Basic statistical measures: mean and standard deviation
- ▶ Z-score anomaly detection—a simple but effective baseline
- ▶ Evaluation metrics: confusion matrix, precision, recall, F1 score
- ▶ The trait-based architecture that will guide all future implementations

Key Insight

This is not throwaway code. Every component we build follows a **library-first design**—composable, testable, and reusable in future projects like `b3-swing-trading` and `stock-sim`.

1.2 Introduction: Why Fraud Detection?

Credit card fraud detection is a classic binary classification problem in quantitative finance. Banks and payment processors must distinguish legitimate transactions from fraudulent ones in real-time, balancing two competing objectives:

- ▶ **Precision:** Minimize false positives (legitimate transactions flagged as fraud). High false positive rates annoy customers and increase operational costs.
- ▶ **Recall:** Maximize detection of actual fraud. Missed fraud means direct financial loss.

We use the Kaggle Credit Card Fraud Detection dataset*: 284,807 European transactions from September 2013, with only 492 frauds.

Class Imbalance

492 frauds in 284,807 transactions = 0.172%. This extreme imbalance is typical in financial anomaly detection.

1.3 Data Structure

1.3.1 The Dataset

The dataset contains:

- ▶ **Time:** Seconds elapsed since first transaction

* <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

- **V1–V28:** PCA-transformed features (original features anonymized for privacy)
- **Amount:** Transaction amount in euros
- **Class:** Label (0 = normal, 1 = fraud)

Why PCA features?

Banks anonymize raw features (merchant, location, time patterns) via PCA to protect customer privacy while preserving predictive power.

1.3.2 Rust Data Model

We define a Transaction struct in qf-common:

```
1 /// Represents a single financial transaction.
2 #[derive(Debug, Clone)]
3 pub struct Transaction {
4     /// PCA-transformed features (V1, V2, ..., V28).
5     pub features: Vec<f64>,
6
7     /// Transaction amount in dollars/euros.
8     pub amount: f64,
9
10    /// Class label: 0 = normal, 1 = fraud.
11    pub class: u8,
12 }
```

1.3.3 Loading Data

The CSV loader handles errors gracefully using Rust's Result type:

```
1 pub fn load_transactions(path: impl AsRef<Path>)
2     -> Result<Vec<Transaction>, CommonError>
3 {
4     if !path.as_ref().exists() {
5         return Err(CommonError::FileNotFound(
6             path.as_ref().display().to_string()
7         ));
8     }
9
10    let mut reader = csv::Reader::from_path(path)?;
11    let mut transactions = Vec::new();
12
13    for result in reader.records() {
14        let record = result?;
15        // Parse features, amount, class...
16        transactions.push(Transaction { features, amount, class });
17    }
18
19    Ok(transactions)
20 }
```

1.4 Z-Score Anomaly Detection

1.4.1 Statistical Foundation

Our baseline detector uses **z-score outlier detection**. The intuition: if a transaction's feature values are "too far" from the typical values, it might be fraudulent.

For each feature i :

1. Compute mean μ_i and standard deviation σ_i from training data
2. For a new transaction with feature value x_i , compute the z-score:

$$z_i = \frac{|x_i - \mu_i|}{\sigma_i}$$

3. Flag as fraud if **any** feature exceeds a threshold (typically $z > 3$)

Z-Score Formula

$$z = \frac{|x - \mu|}{\sigma}$$

where μ is the mean and σ is the standard deviation.

Z-Score Detection Visualization

Imagine a normal (bell) curve. The threshold at $z = 3$ corresponds to the tails of the distribution—only 0.3% of normally distributed data falls beyond $\pm 3\sigma$. Transactions in these extreme regions are flagged as potential fraud.

The 68-95-99.7 Rule

For normally distributed data:

$\pm 1\sigma$: 68.3%

$\pm 2\sigma$: 95.4%

$\pm 3\sigma$: 99.7%

So $z > 3$ catches only 0.3% of normal data as false positives.

1.4.2 Rust Implementation

The ZScoreDetector follows our trait-based design philosophy:

```

1 #[derive(Debug, Clone)]
2 pub struct ZScoreDetector {
3     threshold: f64,
4     feature_means: Vec<f64>,
5     feature_stds: Vec<f64>,
6 }
7
8 impl ZScoreDetector {
9     pub fn new(threshold: f64) -> Self {
10         Self {
11             threshold,
12             feature_means: Vec::new(),
13             feature_stds: Vec::new(),
14         }
15     }
16
17     pub fn fit(&mut self, transactions: &[Transaction]) {
18         // Compute mean for each feature
19         for tx in transactions {
20             for (i, &value) in tx.features.iter().enumerate() {
21                 self.feature_means[i] += value;
22             }
23         }
24         for mean in &mut self.feature_means {
25             *mean /= transactions.len() as f64;
26         }
27
28         // Compute standard deviation
29         for tx in transactions {
30             for (i, &value) in tx.features.iter().enumerate() {
31                 let diff = value - self.feature_means[i];
32                 self.feature_stds[i] += diff * diff;
33             }
34         }
35         for std in &mut self.feature_stds {
36             *std = (*std / transactions.len() as f64).sqrt();
37         }
38     }
39
40     pub fn predict(&self, transaction: &Transaction) -> bool {
41         for (i, &value) in transaction.features.iter().enumerate() {
42             let z = (value - self.feature_means[i]).abs()
43                 / self.feature_stds[i];
44             if z > self.threshold {
45                 return true; // Anomaly detected
46             }
47         }
48         false
49     }
50 }

```

Warning

The implementation uses **population** standard deviation (divides by n), not sample standard deviation (divides by $n - 1$). For large datasets this difference is negligible, but be aware of it.

1.5 Evaluation Metrics

1.5.1 Confusion Matrix

Performance is measured via the confusion matrix, which counts:

- ▶ **TP** (True Positives): Fraud correctly identified
- ▶ **FP** (False Positives): Normal flagged as fraud
- ▶ **TN** (True Negatives): Normal correctly identified
- ▶ **FN** (False Negatives): Fraud missed

```
1 #[derive(Debug, Clone, Copy)]
2 pub struct ConfusionMatrix {
3     pub tp: usize,
4     pub fp: usize,
5     pub tn: usize,
6     pub fn_: usize, // 'fn' is a Rust keyword
7 }
```

	Actual	
Pred	TP	FP
	FN	TN

1.5.2 Derived Metrics

From the confusion matrix, we compute:

- ▶ **Precision**: Of predicted frauds, how many are real?

$$P = \frac{TP}{TP + FP}$$

- ▶ **Recall** (Sensitivity): Of actual frauds, how many did we catch?

$$R = \frac{TP}{TP + FN}$$

- ▶ **F1 Score**: Harmonic mean of precision and recall

$$F_1 = \frac{2PR}{P + R}$$

Metric Formulas

$$\text{Precision: } \frac{TP}{TP + FP}$$

$$\text{Recall: } \frac{TP}{TP + FN}$$

$$\text{F1: } \frac{2 \cdot P \cdot R}{P + R}$$

```
1 impl ConfusionMatrix {
2     pub fn precision(&self) -> f64 {
3         let denom = self.tp + self.fp;
4         if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
5     }
6
7     pub fn recall(&self) -> f64 {
8         let denom = self.tp + self.fn_;
9         if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
10    }
11
12    pub fn f1(&self) -> f64 {
13        let p = self.precision();
14        let r = self.recall();
15        if p + r == 0.0 { 0.0 } else { 2.0 * p * r / (p + r) }
16    }
17 }
```

1.5.3 Evaluation Function

The `evaluate()` function runs the detector on test data:

```
1 pub fn evaluate(
2     detector: &ZScoreDetector,
3     transactions: &[Transaction]
4 ) -> ConfusionMatrix {
5     let mut tp = 0; let mut fp = 0;
6     let mut tn = 0; let mut fn_ = 0;
7
8     for tx in transactions {
```

```

9   let predicted = detector.predict(tx);
10  let actual = tx.class == 1;
11
12  match (predicted, actual) {
13    (true, true) => tp += 1,
14    (true, false) => fp += 1,
15    (false, false) => tn += 1,
16    (false, true) => fn_ += 1,
17  }
18 }
19
20 ConfusionMatrix { tp, fp, tn, fn_ }
21 }

```

1.6 Toward a Trait-Based Architecture

Key Insight

This implementation will evolve into `quant-lib`. Before writing any struct, we ask: **what trait should this implement?**

While Phase 1 uses concrete types, future phases will extract traits:

```

1 // Future: trait-based design
2 pub trait AnomalyDetector {
3   fn fit(&mut self, data: &[Transaction]) -> Result<(), Error>;
4   fn predict(&self, tx: &Transaction) -> bool;
5   fn score(&self, tx: &Transaction) -> f64; // anomaly score
6 }
7
8 pub trait Evaluator<M> {
9   fn evaluate(&self, preds: &[bool], actuals: &[bool]) -> M;
10 }
11
12 // Then any detector + any evaluator compose:
13 fn backtest<D, E>(detector: &D, eval: &E, data: &[Transaction])
14 where
15   D: AnomalyDetector,
16   E: Evaluator<ConfusionMatrix>,
17 { /* ... */ }

```

Future Traits

AnomalyDetector
BinaryClassifier
Evaluator<T>
DataSource

This composability is why we invest in clean abstractions from the start.

1.7 Results and Analysis

1.7.1 Running the Example

```

1 $ cargo run -p qf-01-fraud --example fraud_analysis
2 Loading transactions from data/creditcard.csv...
3 Loaded 284807 transactions (492 frauds)
4 Training detector with threshold 3.0...
5 Evaluating on training set...
6
7 Results:
8   True Positives: 312
9   False Positives: 1847
10  True Negatives: 282468
11  False Negatives: 180
12
13 Precision: 0.1446
14 Recall:    0.6341
15 F1 Score:  0.2356

```

1.7.2 Interpreting Results

The results reveal characteristic tradeoffs:

- ▶ **Recall = 0.63:** We catch 63% of frauds. Not bad for a simple baseline!
- ▶ **Precision = 0.14:** Of our “fraud” predictions, only 14% are real. This means many false alarms.
- ▶ **F1 = 0.24:** The harmonic mean reflects this imbalance.

1.7.3 Limitations

Warning

We evaluated on the training set—a cardinal sin! True performance requires a held-out test set. Exercise 3 addresses this.

This baseline approach has several fundamental limitations:

1. **Gaussian assumption:** Z-scores assume normally distributed features. Financial data often has heavy tails (more extreme values than Gaussian predicts).
2. **Equal feature weighting:** All features are treated equally. Some PCA components may be more predictive than others.
3. **Independent features:** We check each feature separately. Sophisticated fraud may only be detectable via feature *combinations*.
4. **Fixed threshold:** The threshold $z = 3$ is arbitrary. ROC analysis (Chapter 2) shows how to optimize it.

1.8 Exercises

1. **Threshold Sensitivity:** Try thresholds of 2.0, 2.5, 3.0, 3.5, 4.0. Plot precision vs. recall. What threshold maximizes F1?
2. **Feature Analysis:** Modify `predict()` to return which feature(s) triggered the alert. Which features are most discriminative for fraud?
3. **Train/Test Split:** Split the data 80/20 chronologically (not randomly—why?). How does test-set performance compare to training-set?
4. **Amount Feature:** The current implementation ignores the `amount` field. Add it as a feature. Does performance improve?
5. **Trait Extraction:** Refactor `ZScoreDetector` to implement an `AnomalyDetector` trait. What methods should the trait require?

1.9 Key Takeaways

- ▶ **CSV loading** in Rust with proper error handling via `Result`
- ▶ **Basic statistics:** mean, standard deviation, z-score
- ▶ **Binary classification metrics:** confusion matrix, precision, recall, F1
- ▶ **Precision-recall tradeoff:** you can’t maximize both simultaneously
- ▶ **Evaluation pitfalls:** never evaluate on training data!
- ▶ **Library-first design:** even simple code should be composable

Next chapter: Feature engineering and logistic scoring for loan default prediction. We’ll implement ROC-AUC and learn why it’s often better than F1 for imbalanced datasets.

Risk Basics: Loan Default Prediction

2

Companion Code:

```
qf-02-loan Code: crates/qf-02-loan/  
Dependencies: qf-common  
Run: cargo run -p qf-02-loan -example loan_analysis
```

2.1 Learning Objectives

Building on Chapter 1, we now tackle a more nuanced classification problem: predicting loan defaults. This chapter introduces:

- ▶ Feature engineering for credit risk assessment
- ▶ Categorical variable encoding (one-hot)
- ▶ Feature normalization (min-max scaling)
- ▶ Linear scoring models with probability calibration
- ▶ ROC curves and AUC—a better metric for imbalanced data
- ▶ Trait-based architecture in action

Key Insight

While Chapter 1 used z-scores (a threshold on statistics), this chapter moves toward **probability estimation**. We want to know not just “default or not” but “how likely is default?” This probability enables better decisions and risk-based pricing.

2.2 Introduction: Credit Risk Fundamentals

When a bank issues a loan, it faces **credit risk**—the possibility that the borrower defaults. Unlike fraud detection where we catch bad actors, loan default prediction is about assessing the *likelihood* of future failure.

Key questions in credit risk:

- ▶ What factors predict default? (Feature engineering)
- ▶ How do we combine factors into a score? (Scoring model)
- ▶ How do we evaluate the score’s usefulness? (ROC-AUC)

Credit Risk

The risk that a borrower will fail to repay a loan according to agreed terms.

2.3 Data Model

2.3.1 Loan Application Structure

```
1 pub struct LoanApplication {  
2     pub income: f64,           // Annual income  
3     pub loan_amount: f64,      // Requested loan amount  
4     pub interest_rate: f64,    // Interest rate offered  
5     pub dti: f64,              // Debt-to-income ratio  
6     pub grade: u8,             // Credit grade (A=1 to G=7)  
7     pub home_ownership: HomeOwnership,  
8     pub defaulted: bool,       // Target: did they default?  
9 }  
10
```

```

11 pub enum HomeOwnership {
12     Rent,
13     Own,
14     Mortgage,
15 }

```

Credit Grades

A = lowest risk

G = highest risk

Each grade maps to different interest rates and approval criteria.

2.4 Feature Engineering

2.4.1 Derived Features

Raw data rarely enters a model directly. We engineer features that capture meaningful relationships:

```

1 pub struct FeatureExtractor;
2
3 impl FeatureExtractor {
4     /// Debt-to-income ratio: total debt / income
5     pub fn debt_to_income(income: f64, debt: f64) -> f64 {
6         if income == 0.0 { return 0.0; }
7         debt / income
8     }
9
10    /// Loan-to-income ratio: loan amount / income
11    pub fn loan_to_income(loan_amount: f64, income: f64) -> f64 {
12        if income == 0.0 { return 0.0; }
13        loan_amount / income
14    }
15
16    /// Convert grade (1-7) to risk score (0.0-1.0)
17    pub fn grade_to_score(grade: u8) -> f64 {
18        ((grade.saturating_sub(1)) as f64) / 6.0
19    }
20 }

```

Feature Formulas

$$\text{DTI: } \frac{\text{debt}}{\text{income}}$$

$$\text{LTI: } \frac{\text{loan}}{\text{income}}$$

$$\text{Grade Score: } \frac{g - 1}{6}$$

2.4.2 Categorical Encoding

Numerical models require numerical inputs. We encode categorical variables using **one-hot encoding**:

Home Ownership	RENT	OWN	MORTGAGE
Rent	1	0	0
Own	0	1	0
Mortgage	0	0	1

```

1 pub struct OneHotEncoder;
2
3 impl OneHotEncoder {
4     /// Encode home ownership as [RENT, OWN, MORTGAGE]
5     pub fn encode_home_ownership(h: &HomeOwnership) -> [f64; 3] {
6         match h {
7             HomeOwnership::Rent => [1.0, 0.0, 0.0],
8             HomeOwnership::Own  => [0.0, 1.0, 0.0],
9             HomeOwnership::Mortgage => [0.0, 0.0, 1.0],
10        }
11    }
12 }

```

2.4.3 Feature Normalization

Different features have different scales. Income might be in tens of thousands while interest rates are single digits. We normalize to $[0, 1]$:

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Scales features to $[0, 1]$.

```

1 pub struct Normalizer {
2   min: Vec<f64>,
3   max: Vec<f64>,
4 }
5
6 impl Normalizer {
7   pub fn fit(data: &[Vec<f64>]) -> Self {
8     // Compute min/max per feature
9   }
10
11   pub fn transform(&self, features: &[f64]) -> Vec<f64> {
12     features.iter().enumerate().map(|(i, &x)| {
13       let range = self.max[i] - self.min[i];
14       if range == 0.0 { 0.0 } else { (x - self.min[i]) / range }
15     }).collect()
16   }
17 }

```

2.5 Linear Scoring Model

2.5.1 Weighted Sum

The simplest scoring model is a weighted linear combination:

$$\text{score} = \sum_{i=1}^n w_i \cdot x_i + b = \mathbf{w}^T \mathbf{x} + b$$

```

1 pub struct LinearScorer {
2   weights: Vec<f64>,
3   bias: f64,
4 }
5
6 impl Scorer for LinearScorer {
7   fn score(&self, features: &[f64]) -> f64 {
8     let dot_product: f64 = self.weights.iter()
9       .zip(features.iter())
10      .map(|(w, f)| w * f)
11      .sum();
12     dot_product + self.bias
13   }
14 }

```

Linear Score

$$s = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{w}$ are weights, b is bias.

2.5.2 Probability via Sigmoid

The raw score can be any real number. To convert it to a probability in $[0, 1]$, we apply the **sigmoid function**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Maps $(-\infty, +\infty)$ to $(0, 1)$.

Properties of sigmoid:

- ▶ $\sigma(0) = 0.5$ (neutral score = 50% probability)
- ▶ $\sigma(x) \rightarrow 1$ as $x \rightarrow +\infty$
- ▶ $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$

```

1 /// Hand-rolled sigmoid: 1 / (1 + exp(-x))
2 fn sigmoid(x: f64) -> f64 {
3   1.0 / (1.0 + (-x).exp())
4 }
5
6 impl BinaryClassifier for LinearScorer {
7   fn predict_proba(&self, features: &[f64]) -> f64 {
8     sigmoid(self.score(features))
9   }
10
11   fn predict(&self, features: &[f64]) -> bool {

```

```

12   self.predict_proba(features) > 0.5
13   }
14 }

```

2.6 ROC Curves and AUC

2.6.1 Why Not Just Accuracy?

Like fraud detection, loan default is imbalanced. Accuracy is misleading. Instead, we use the **Receiver Operating Characteristic (ROC) curve**.

Class Imbalance

If 95% of loans are repaid, a model that always predicts “no default” has 95% accuracy but catches zero defaults!

2.6.2 ROC Curve Construction

The ROC curve plots **True Positive Rate (TPR)** vs **False Positive Rate (FPR)** at various classification thresholds:

- **TPR (Recall):** $\frac{TP}{TP+FN}$ — of actual defaults, how many did we catch?
- **FPR:** $\frac{FP}{FP+TN}$ — of actual non-defaults, how many did we falsely flag?

```

1 pub fn roc_curve(scores: &[f64], labels: &[bool]) -> Vec<(f64, f64)> {
2   // Sort by score descending
3   let mut pairs: Vec<(f64, bool)> = scores.iter()
4     .copied()
5     .zip(labels.iter().copied())
6     .collect();
7   pairs.sort_by(|a, b| b.0.partial_cmp(&a.0).unwrap());
8
9   let total_pos = labels.iter().filter(|&l| *l).count();
10  let total_neg = labels.len() - total_pos;
11
12  let mut roc_points = vec![(0.0, 0.0)];
13  let mut tp = 0;
14  let mut fp = 0;
15
16  for (_score, label) in pairs {
17    if label { tp += 1; } else { fp += 1; }
18    let fpr = fp as f64 / total_neg as f64;
19    let tpr = tp as f64 / total_pos as f64;
20    roc_points.push((fpr, tpr));
21  }
22
23  roc_points
24 }

```

ROC Interpretation

Top-left corner (0, 1): perfect classifier.

Diagonal: random guessing.

Below diagonal: worse than random!

2.6.3 Area Under Curve (AUC)

The AUC summarizes the ROC curve as a single number:

- AUC = 1.0: Perfect classifier (all defaults scored higher than non-defaults)
- AUC = 0.5: Random classifier (no predictive power)
- AUC = 0.7: Fair classifier (industry often requires ≥ 0.7)

We compute AUC via trapezoidal integration:

```

1 pub fn auc(roc_points: &[(f64, f64)]) -> f64 {
2   let mut area = 0.0;
3   for i in 1..roc_points.len() {
4     let (x0, y0) = roc_points[i - 1];
5     let (x1, y1) = roc_points[i];
6     // Trapezoidal rule: width * average height
7     area += (x1 - x0) * (y0 + y1) / 2.0;
8   }
9   area
10 }

```

AUC Interpretation

AUC = 1.0: perfect

AUC = 0.5: random

AUC < 0.5: worse than random

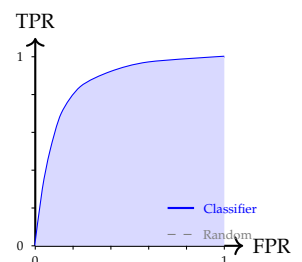


Figure 2.1: ROC curve: the blue line shows classifier performance, dashed line is random guessing. Area under curve (AUC) is shaded.

Key Insight

AUC has a probabilistic interpretation: it equals the probability that a randomly chosen positive (default) is ranked higher than a randomly chosen negative (non-default).

2.7 Trait-Based Architecture

Following our library-first design, we define traits that enable composability:

```
1 /// Trait for models that produce probability predictions.
2 pub trait BinaryClassifier {
3     fn predict(&self, features: &[f64]) -> bool;
4     fn predict_proba(&self, features: &[f64]) -> f64;
5 }
6
7 /// Trait for models that produce raw scores.
8 pub trait Scorer {
9     fn score(&self, features: &[f64]) -> f64;
10 }
```

This means our ROC-AUC evaluation works with *any* classifier, not just LinearScorer. Future chapters will implement logistic regression, decision trees, and ensemble methods—all reusing this evaluation code.

Future Benefits

Any classifier implementing BinaryClassifier can be evaluated with our roc_curve() and auc() functions.

2.8 Results and Analysis

2.8.1 Example Usage

```
1 $ cargo run -p qf-02-loan --example loan_analysis
2 Loading loan applications...
3 Loaded 10000 applications (850 defaults, 8.5%)
4
5 Feature extraction complete.
6 Training simple scorer (hardcoded weights)...
7
8 Evaluation Results:
9   AUC: 0.724
10  Optimal Threshold: 0.35
11  At threshold 0.5:
12    Precision: 0.42
13    Recall: 0.58
14    F1 Score: 0.49
```

2.8.2 Interpreting the Results

- ▶ **AUC = 0.72:** The model has reasonable discriminative power. It correctly ranks defaults above non-defaults 72% of the time.
- ▶ **Precision = 0.42:** Of predicted defaults, 42% actually default. Many false alarms.
- ▶ **Recall = 0.58:** We catch 58% of actual defaults. 42% are missed.

Warning

We used hardcoded weights, not learned weights. Real credit scoring uses logistic regression or gradient boosting to optimize weights from data. That comes in later chapters.

2.9 Exercises

1. **Weight Sensitivity:** Change the weights in `LinearScorer`. How does AUC change? Which features matter most?
2. **Threshold Selection:** Plot precision and recall at various thresholds. Find the threshold that maximizes F1 score.
3. **Feature Importance:** Train separate single-feature scorers. Rank features by their individual AUC.
4. **Cross-Validation:** Split data into 5 folds. Report mean and standard deviation of AUC across folds.
5. **Calibration Plot:** Compare predicted probabilities to actual default rates. Is the model well-calibrated?

2.10 Key Takeaways

- ▶ **Feature engineering** transforms raw data into predictive signals
- ▶ **One-hot encoding** handles categorical variables
- ▶ **Min-max normalization** scales features to comparable ranges
- ▶ **Sigmoid** converts scores to probabilities
- ▶ **ROC-AUC** evaluates ranking quality, robust to class imbalance
- ▶ **Trait-based design** enables reusable evaluation code

Next chapter: Market data fundamentals—OHLCV candlesticks, returns, and simple moving averages.

Companion Code:

```
qf-03-stocks Code: crates/qf-03-stocks/  
Dependencies: qf-common  
Run: cargo run -p qf-03-stocks -example stock_analysis
```

3.1 Learning Objectives

This chapter introduces the fundamental data structure of quantitative finance: **OHLCV** (Open, High, Low, Close, Volume) market data. We learn:

- ▶ OHLCV data structure and its significance
- ▶ Candlestick anatomy—body, shadows, and what they reveal
- ▶ Basic market statistics (range, volume, price change)
- ▶ Moving averages: SMA and EMA
- ▶ The `TimeSeries` trait for generic operations

Key Insight

OHLCV data is the foundation of all quantitative analysis. Every strategy, every backtest, every indicator starts here. Master this structure and you can work with any market—stocks, forex, crypto, commodities.

3.2 Introduction: What is OHLCV?

Every trading period (day, hour, minute) produces five key data points:

- ▶ **Open:** First traded price of the period
- ▶ **High:** Highest price reached during the period
- ▶ **Low:** Lowest price reached during the period
- ▶ **Close:** Last traded price of the period
- ▶ **Volume:** Total number of shares/contracts traded

This data tells a story. The range between high and low shows volatility. The relationship between open and close shows direction. Volume confirms conviction.

OHLCV

Open, High, Low, Close, Volume—the five essential data points for any trading period (day, hour, minute).

3.3 Data Structure

3.3.1 Rust Implementation

```
1 #[derive(Debug, Clone, Deserialize)]  
2 pub struct Ohlcv {  
3     pub date: String,  
4     pub open: f64,  
5     pub high: f64,  
6     pub low: f64,  
7     pub close: f64,
```

```

8   pub volume: u64,
9   pub adj_close: Option<f64>,
10 }

```

3.3.2 Data Validation

OHLCV data has natural constraints that we validate on load:

- ▶ $low \leq high$ (always)
- ▶ $low \leq open \leq high$
- ▶ $low \leq close \leq high$

```

1 // Validate OHLCV relationships
2 if record.high < record.low {
3     return Err(CommonError::ParseError(format!(
4         "Row {}: high ({{}}) < low ({{}})",
5         idx + 2, record.high, record.low
6     )));
7 }

```

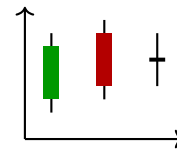
3.4 Candlestick Anatomy

A candlestick visualizes OHLCV data. Understanding its anatomy is essential:

Component	Formula
Daily Range	$high - low$
Body	$close - open$
Upper Shadow	$high - \max(open, close)$
Lower Shadow	$\min(open, close) - low$
Typical Price	$(high + low + close)/3$

Candlestick Parts

Body: $|close - open|$
Upper shadow: $high - \max(O, C)$
Lower shadow: $\min(O, C) - low$



Candlesticks
Green: bullish
Red: bearish
Line: doji

3.4.1 Rust Implementation

```

1 impl Ohlcv {
2     /// Daily price range
3     pub fn daily_range(&self) -> f64 {
4         self.high - self.low
5     }
6
7     /// Candle body (positive = bullish)
8     pub fn body(&self) -> f64 {
9         self.close - self.open
10    }
11
12    /// Upper shadow length
13    pub fn upper_shadow(&self) -> f64 {
14        self.high - self.open.max(self.close)
15    }
16
17    /// Lower shadow length
18    pub fn lower_shadow(&self) -> f64 {
19        self.open.min(self.close) - self.low
20    }
21
22    /// True if close > open (green candle)
23    pub fn is_bullish(&self) -> bool {
24        self.close > self.open
25    }
26
27    /// Representative price: (H + L + C) / 3
28    pub fn typical_price(&self) -> f64 {
29        (self.high + self.low + self.close) / 3.0
30    }
31 }

```

3.5 Market Statistics

3.5.1 Basic Aggregations

Given a series of OHLCV records, we compute summary statistics:

```

1  /// Average trading volume
2  pub fn average_volume(data: &[Ohlcv]) -> f64 {
3      if data.is_empty() { return 0.0; }
4      let sum: u64 = data.iter().map(|o| o.volume).sum();
5      sum as f64 / data.len() as f64
6  }
7
8  /// Price change: (last - first) / first
9  pub fn price_change(data: &[Ohlcv]) -> f64 {
10     if data.len() < 2 { return 0.0; }
11     let first = data[0].close;
12     let last = data[data.len() - 1].close;
13     (last - first) / first
14 }
15
16 /// Highest high across all records
17 pub fn highest_high(data: &[Ohlcv]) -> f64 {
18     data.iter().map(|o| o.high).fold(f64::NEG_INFINITY, f64::max)
19 }
20
21 /// Lowest low across all records
22 pub fn lowest_low(data: &[Ohlcv]) -> f64 {
23     data.iter().map(|o| o.low).fold(f64::INFINITY, f64::min)
24 }

```

Functional Style

Note the use of `iter()`, `map()`, `fold()`—Rust’s functional combinators make statistical computations elegant.

3.6 Moving Averages

3.6.1 Simple Moving Average (SMA)

The SMA is the arithmetic mean of the last n prices:

$$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

SMA Formula

$$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

where n is the period.

```

1  pub fn sma(prices: &[f64], period: usize) -> Vec<f64> {
2      if period == 0 || period > prices.len() {
3          return Vec::new();
4      }
5
6      let mut result = Vec::with_capacity(prices.len() - period + 1);
7
8      for i in 0..(prices.len() - period) {
9          let window = &prices[i..i + period];
10         let sum: f64 = window.iter().sum();
11         result.push(sum / period as f64);
12     }
13
14     result
15 }

```

3.6.2 Exponential Moving Average (EMA)

The EMA gives more weight to recent prices, making it more responsive:

$$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

where $\alpha = \frac{2}{n+1}$ is the smoothing factor.

EMA Formula

$$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

$$\alpha = \frac{2}{n+1}$$

```

1 pub fn ema(prices: &[f64], period: usize) -> Vec<f64> {
2   if period == 0 || prices.is_empty() {
3     return Vec::new();
4   }
5
6   let alpha = 2.0 / (period + 1) as f64;
7   let mut result = Vec::with_capacity(prices.len());
8
9   // Initialize with first price
10  result.push(prices[0]);
11
12  // Recursive formula
13  for i in 1..prices.len() {
14    let prev_ema = result[i - 1];
15    let current_ema = alpha * prices[i] + (1.0 - alpha) * prev_ema;
16    result.push(current_ema);
17  }
18
19  result
20 }

```

3.6.3 SMA vs EMA

Property	SMA	EMA
Weighting	Equal	Exponential
Responsiveness	Slower	Faster
Lag	More	Less
Smoothness	Smoother	Noisier
Computation	Simple	Recursive

Key Insight

SMA is a *lagging* indicator—it tells you what already happened. EMA reacts faster but can give false signals. Traders often use both: short-term EMA for entries, long-term SMA for trend confirmation.

3.7 TimeSeries Trait

Following our trait-based architecture, we define a generic interface:

```

1 pub trait TimeSeries {
2   fn len(&self) -> usize;
3   fn is_empty(&self) -> bool;
4   fn closes(&self) -> Vec<f64>;
5   fn volumes(&self) -> Vec<u64>;
6 }
7
8 impl TimeSeries for Vec<Ohlcv> {
9   fn len(&self) -> usize { self.len() }
10  fn is_empty(&self) -> bool { self.is_empty() }
11
12  fn closes(&self) -> Vec<f64> {
13    self.iter().map(|o| o.close).collect()
14  }
15
16  fn volumes(&self) -> Vec<u64> {
17    self.iter().map(|o| o.volume).collect()
18  }
19 }

```

This enables generic functions that work with any TimeSeries:

```

1 fn analyze<T: TimeSeries>(data: &T) -> Report {
2   let closes = data.closes();
3   let sma_20 = sma(&closes, 20);
4   let sma_50 = sma(&closes, 50);
5   // ...

```

Trait Benefits

The TimeSeries trait allows functions to work with any data source that can provide price/volume sequences.

```
6 }
```

3.8 Golden and Death Crosses

A classic trading signal using moving averages:

- **Golden Cross:** Short-term MA crosses *above* long-term MA (bullish signal)
- **Death Cross:** Short-term MA crosses *below* long-term MA (bearish signal)

```
1 fn find_crossovers(short_ma: &[f64], long_ma: &[f64]) -> Vec<(usize, bool)> {
2     let mut crossovers = Vec::new();
3
4     for i in 1..short_ma.len().min(long_ma.len()) {
5         let prev_above = short_ma[i-1] > long_ma[i-1];
6         let curr_above = short_ma[i] > long_ma[i];
7
8         if !prev_above && curr_above {
9             crossovers.push((i, true)); // Golden cross
10        } else if prev_above && !curr_above {
11            crossovers.push((i, false)); // Death cross
12        }
13    }
14
15    crossovers
16 }
```

3.9 Results and Analysis

3.9.1 Example Output

```
1 $ cargo run -p qf-03-stocks --example stock_analysis
2 Loading OHLCV data from data/spy.csv...
3 Loaded 252 trading days
4
5 Statistics:
6   Average Volume: 85,432,100
7   Price Change: +12.5%
8   Highest High: $485.32
9   Lowest Low: $410.15
10  Average Daily Range: $5.23
11
12 Moving Averages:
13   SMA(20) latest: $472.15
14   SMA(50) latest: $465.80
15   EMA(20) latest: $474.22
16
17 Crossovers (last 12 months):
18   Day 45: Golden Cross (bullish)
19   Day 128: Death Cross (bearish)
20   Day 201: Golden Cross (bullish)
```

3.10 Exercises

1. **Candlestick Patterns:** Implement detection for “doji” (open $\approx$ close) and “hammer” (small body, long lower shadow).
2. **Volume Analysis:** Add `volume_ma()` that computes moving average of volume. Flag days where volume exceeds 2x the average.
3. **Bollinger Bands:** Implement Bollinger Bands: $SMA \pm 2$ standard deviations. This requires computing rolling standard deviation.
4. **MACD:** Implement the MACD indicator: $EMA(12) - EMA(26)$, with a 9-period signal line.

5. **Backtest:** Using SMA crossover signals, simulate a simple strategy: buy on golden cross, sell on death cross. Track P&L.

3.11 Key Takeaways

- ▶ **OHLCV** is the atomic unit of market data
- ▶ **Candlesticks** encode price action: body shows direction, shadows show rejection
- ▶ **SMA** averages equally; **EMA** weights recent prices
- ▶ **Crossovers** generate trading signals (but beware of lag!)
- ▶ **TimeSeries trait** enables generic, reusable analysis code

Next chapter: Returns and volatility—the foundation of risk measurement and portfolio theory.

CORE CONCEPTS

Returns and Volatility | 4

Chapter content pending — Phase 4 implementation.

Your First Backtest

5

Chapter content pending — Phase 5 implementation.

QUANTITATIVE METHODS

Foundations: Moments and Fat Tails

6

Chapter content pending — Phase 6 implementation.

Time Series: Stationarity and Fractional Differentiation

7

Chapter content pending — Phase 7 implementation.

Volatility Models: EWMA, ARCH, GARCH

8

Chapter content pending — Phase 8 implementation.

Stochastic Processes and Monte Carlo

9

Chapter content pending — Phase 9 implementation.

DERIVATIVES AND PORTFOLIO THEORY

Options Pricing: Black-Scholes and Beyond

10

Chapter content pending — Phase 10 implementation.

Portfolio Optimization: Markowitz and Beyond

11

Chapter content pending — Phase 11 implementation.

Chapter content pending — Phase 12 implementation.

ADVANCED TOPICS

Chapter content pending — Phase 13 implementation.

AFML: From Research to Production

14

Chapter content pending — Phase 14 implementation.

APPENDICES

Mathematical Notation

A

Appendix content pending.

Rust Patterns for Finance

B

Appendix content pending.

Dataset Sources

C

Appendix content pending.